B.1. Logit lens pathologies

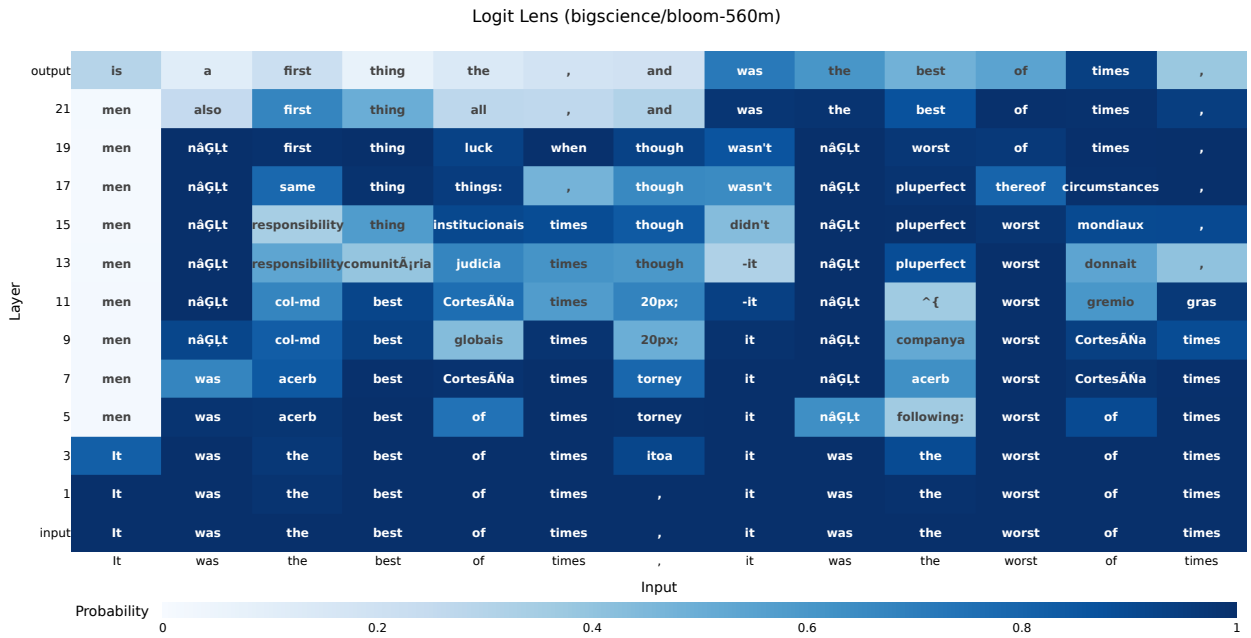

Figure 17. Logit lens prediction trajectory for BLOOM 560M. The logit lens assigns as very high probability to the *input* token at many layers and token positions, complicating the interpretation of the output.

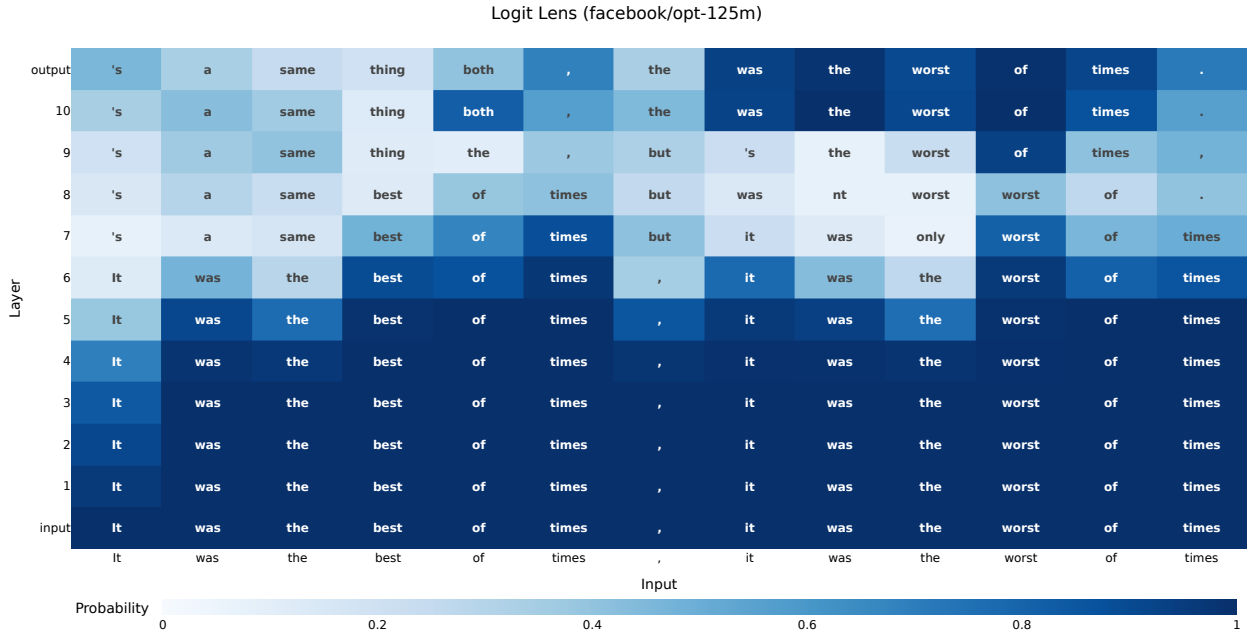

Figure 18. Logit lens prediction trajectory for OPT 125M, exhibiting a similar pathology to BLOOM 560M above.

C. Transformers Perform Iterative Inference

Jastrzębski et al. (2017) argue that skip connections encourage neural networks to perform "iterative inference" in the following sense: each layer reliably updates the hidden state in a direction of decreasing loss. While their analysis focused on ResNet image classifiers, their theoretical argument applies equally well to transformers, or any neural network featuring residual connections. We reproduce their theoretical analysis below.

A residual block applied to a representation $\boldsymbol{h}_i$ updates the representation as follows:

$$\boldsymbol{h}_{\ell+1} = \boldsymbol{h}_\ell + F_\ell(\boldsymbol{h}_\ell)$$

Let $\mathcal{L}$ denote the final linear classifier followed by a loss function. We can Taylor expand $\mathcal{L}(\boldsymbol{h}_L)$ around $\boldsymbol{h}_i$ to yield

$$\mathcal{L}(\boldsymbol{h}_L) = \mathcal{L}(\boldsymbol{h}_i) + \underbrace{\sum_{j=i}^{L} \left\langle F_j(\boldsymbol{h}_j), \frac{\partial \mathcal{L}(\boldsymbol{h}_j)}{\partial \boldsymbol{h}_j} \right\rangle}_{\text{gradient-residual alignment}} + \mathcal{O}(F_j^2(\boldsymbol{h}_j)) \tag{19}$$

Thus to a first-order approximation, the model is encouraged to minimize the inner product between the residual $F(\boldsymbol{h}_i)$ and the gradient $\frac{\partial \mathcal{L}(\boldsymbol{h}_i)}{\partial \boldsymbol{h}_i}$, which can be achieved by aligning it with the negative gradient.

To empirically measure the extent to which residuals do align with the negative gradient, Jastrzębski et al. (2017) compute the cosine similarity between $F(\boldsymbol{h}_i)$ and $\frac{\partial \mathcal{L}(\boldsymbol{h}_i)}{\partial \boldsymbol{h}_i}$ at each layer of a ResNet image classifier. They find that it is consistently negative, especially in the final stage of the network.

We reproduced their experiment using Pythia 6.9B, and report the results in Figure 19. We show that, for every layer in the network, the cosine similarity between the residual and the gradient is negative at least 95% of the time. While the magnitudes of the cosine similarities are relatively small in absolute terms, never exceeding 0.05, we show that they are much larger than would be expected of random vectors in this very high dimensional space. Specifically, we first sample 250 random Gaussian vectors of the same dimensionality as $\frac{\partial \mathcal{L}(\boldsymbol{h}_i)}{\partial \boldsymbol{h}_i}$, which is (hidden size) $\times$ (sequence length) $= 8{,}388{,}608$. We then compute pairwise cosine similarities between the vectors, and find the 5th percentile of this sample to be $\mathbf{-6 \times 10^{-4}}$. Virtually all of the gradient-residual pairs we observed had cosine similarities below this number.

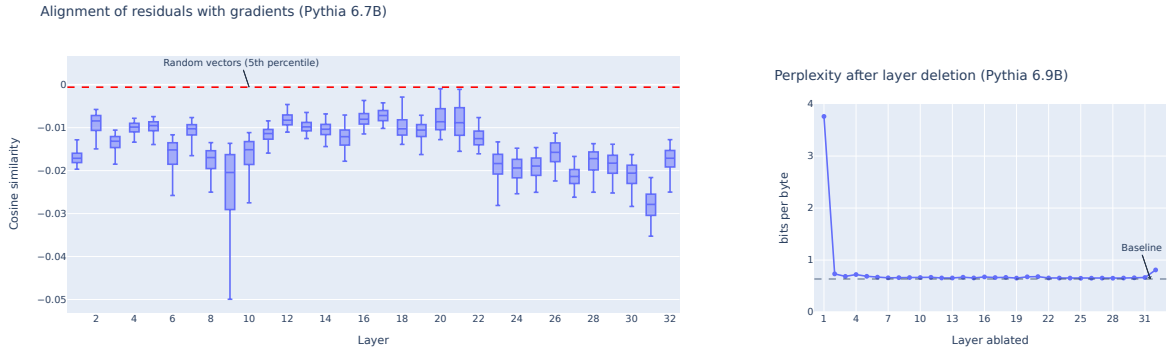

Figure 19. **Left:** Cosine similarity between the observed update to the hidden state and $\frac{\partial \mathcal{L}(\boldsymbol{h}_i)}{\partial \boldsymbol{h}_i}$. The similarity is almost always negative, and of much larger magnitude than would be expected by chance in this very high dimensional space. Boxplot whiskers are 5th and 95th percentiles. Results were computed over a sample of roughly 16M tokens from the Pile validation set. **Right:** Cross-entropy loss of Pythia 6.9B on the Pile validation set, after replacing each of its 32 layers with the identity transformation. The dashed line indicates Pythia's perplexity on this dataset with all layers intact.

C.1. Zero-shot robustness to layer deletion

Stochastic depth (Huang et al., 2016), also known as LayerDrop (Fan et al., 2019), is a regularization technique which randomly drops layers during training, and can be applied to both ResNets and transformers. Veit et al. (2016) show that

ResNets are robust to the deletion of layers even when trained without stochastic depth, while CNN architectures without skip connections are not robust in this way. These results strongly support the idea that adjacent layers in a ResNet encode fundamentally similar representations (Greff et al., 2016).

To the best of our knowledge, this experiment had never been replicated before in transformers. We did so and report our results in Figure 19 above. We find that only the very first layer is crucial for performance; every other layer ablation induces a nearly imperceptible increase in perplexity. Interestingly, Veit et al. (2016) also found that the first layer is exceptionally important in ResNets, suggesting that this is a general phenomenon.

D. Causal intervention details

D.1. Ablation techniques

Prior work has assumed that, given a representation $\boldsymbol{x}$ and a linear subspace $B \subseteq \mathbb{R}^D$ along which we want to remove information, the best erasure strategy is to project $\boldsymbol{x}$ onto the orthogonal complement of B:

$$\boldsymbol{x}' = \mathsf{proj}_{B_\perp}(\boldsymbol{x}) \tag{20}$$

This "zeroes out" a part of $\boldsymbol{x}$, ensuring that $\langle \boldsymbol{x}', \boldsymbol{u} \rangle = \mathbf{0}$ for all $\boldsymbol{u}$ in B. But recent work has pointed out that setting activations to zero may inadvertently push a network out of distribution, making the experimental results hard to interpret (Wang et al., 2022; Chan et al., 2022).

For example, a probe may rely on the invariant that $\forall \boldsymbol{x}, \langle \boldsymbol{x}, \boldsymbol{u} \rangle \gg \mathbf{0}$ for some $\boldsymbol{u}$ in B as an implicit bias term, even if $\langle \boldsymbol{x}, \boldsymbol{u} \rangle$ has low variance and does not encode any useful information about the input. The naive projection $\boldsymbol{x}' = \mathsf{proj}_{B_\perp}(\boldsymbol{x})$ would significantly increase the loss, making it seem that $\mathsf{proj}_B \boldsymbol{x}$ contains important information– when in reality model performance would not be significantly degraded if $\mathsf{proj}_B \boldsymbol{x}$ were replaced with a nonzero constant value more representative of the training distribution.

To correct for this, in our causality experiments we ablate directions by replacing them with their *mean* values computed across a dataset, instead of zeroing them out. Specifically, to ablate a direction $\boldsymbol{u}$, we use the formula:

$$\boldsymbol{x}' = \boldsymbol{x} + P_{\boldsymbol{u}}(\overline{\boldsymbol{x}} - \boldsymbol{x}) \tag{21}$$

where P_u is the projection matrix for $\boldsymbol{u}$ and $\overline{\boldsymbol{x}}$ is the mean representation.

E. Static interpretability analysis

Several interpretability methods aim to analyze parameters that "write" (in the sense of Elhage et al. (2021)) to intermediate hidden states of a language model (Millidge and Black, 2022; Dar et al., 2022; Geva et al., 2022; 2020), and even use this analysis to successfully edit model behavior (Geva et al., 2022; Millidge and Black, 2022; Dai et al., 2022). Given that the tuned lens aims to provide a "less biased" view of intermediate hidden states, we should expect the tuned lens to preserve their effectiveness. We test both static analysis and model editing methods, and confirm this hypothesis. With static analysis, the tuned lens appears to never decrease performance, and for some models, even increases their performance. With model editing, we found the tuned lens to outperform the logit lens on OPT-125m and perform equivalently on other models for the task of toxicity reduction.

E.1. Static Analysis

Many parameters in transformer language models have at least one dimension equal to that of their hidden state, allowing us to multiply by the unembedding matrix to project them into token space. Surprisingly, the resulting top k tokens are often *meaningful*, representing interpretable semantic and syntactic clusters, such as "prepositions" or "countries" (Millidge and Black, 2022; Dar et al., 2022; Geva et al., 2022). This has successfully been applied to the columns of the MLP output matrices (Geva et al., 2022), the singular vectors of the MLP input and output matrices (Millidge and Black, 2022), and the singular vectors of the attention output-value matrices W_{OV} (Millidge and Black, 2022).

In short[7], we can explain this as: many model parameters directly modify the model's hidden state (Elhage et al., 2021), and

[7]The exact interpretation depends on the method used - for instance, the rows of W_{out} can be interpreted as the values in a key-value memory store (Geva et al., 2020).